Access Control & Authentication

Access control overview

Don't trust some programmers

Namespaces

- aka packages, modules

API with public/private

Don't trust some users

Restrict access to DB objects or actions based upon role.

Separate data into different tables with different access.

GRANT, DENY, REVOKE, …

Creating users & roles

```
CREATE USER thisuser PASSWORD 'yourpassword';
CREATE DATABASE mydatabase OWNER thisuser;
```

Role = (group of) users:

```
CREATE ROLE thisrole;
GRANT thisrole TO thisuser;
```

Two levels of namespaces

Databases:

```
CREATE DATABASE mydb OWNER myuser;
```

Schemas:

```
CREATE SCHEMA myschema;
```

Using namespaces:

```
CREATE TABLE myschema.MyTable …;
CREATE TABLE mydatabase.myschema.MyTable …;
```

Default schema name:

- PostgreSQL: 'public'
- SQL Server: 'dbo'
- Oracle: Username
- MySQL: No schemas!

Default privileges in PostgreSQL

- postgres is a superuser – can do anything.
- DB owner has all privileges in that database.
- PUBLIC can execute functions, but little else.

Managing user privileges

```
GRANT UPDATE ON MyTable TO PUBLIC;
GRANT INSERT ON ALL TABLES IN SCHEMA myschema TO user1;
GRANT SELECT (column3) ON MyTable TO user2;
GRANT EXECUTE ON FUNCTION MyFunc(INT) TO role3;
GRANT CREATE ON SCHEMA myschema TO role4;
GRANT CREATE ON DATABASE mydatabase TO user5 WITH GRANT OPTION;

REVOKE ALL PRIVILEGES ON MyTable TO user6;
```

psql command: \dp

Demo: pgAdmin

- Show how to display and change privileges.

Other access control options

- Limit physical access
- Limit network access
- Limit server access
- Limit connections
  - Number, source, duration, time, …
- Encrypt communications & storage